Spot the Fake Photo — Submission Note

Approach

I chose a classical signal-based pipeline over a trained CNN because it meets all key deployment constraints: **<20ms compute**, **<1MB on-device storage**, and **zero serving cost** — no GPU, no API, no inference server.

The pipeline extracts **eight independent signals** from each image and fuses them with a trained weighted sum, then compares the result to a calibrated threshold:

- **FFT peak energy** – detects periodic RGB sub-pixel grids in screens via sharp Fourier spikes.
- **Edge density** – screens contain significantly more edges (UI, text, sub-pixel boundaries); Cohen's d = 2.27 on validation.
- **Color gamut shift** – measures LED backlight cyan-blue cast and screen over-sharpening.
- **Screen luminance signature** – combines blown highlights, mean saturation, and brightness std.
- **Channel decorrelation** – RGB channels are less correlated in screen-emitted light than in natural reflections.
- **Halftone detection** – a separate lower-frequency FFT ring catches printed materials.
- **Glare blob smoothness** – screen glare is smoother than real specular highlights.
- **Double JPEG compression** – detects periodic artifacts in DCT histograms from re-encoding.

Signal weights are optimised via Dirichlet random search with local refinement, and the threshold is chosen by maximising Youden's J statistic.

Accuracy

90.1% balanced accuracy on a held-out cross-validation of 175 personally captured photos (90 real, 85 screen), verified with 5-fold stratified CV.

- **Confusion matrix**: 84 TN, 73 TP, 6 FP, 11 FN
- **Metrics**: Precision 92.4%, Recall 86.9%, F1 0.896

> [!NOTE] **Honest caveat**: Two signals (`fft_peaks`, `double_compression`) show dataset-specific fitting; the four stable signals (`edge_density`, `color_gamut`, `screen_luminance`, `channel_decorrelation`) generalise more reliably.

Latency & Cost

- **On-device inference** (the intended deployment model): **~50–60ms total** on Streamlit (laptop CPU); signal computation alone is ~20ms; the rest is I/O.
- **Cost per image: $0** — pure NumPy + OpenCV + SciPy, no server, no model weights, no API calls. Because it requires no external services, it can run entirely on the user's device.

Future Improvements

- **More diverse training data**: Target hard negatives (bright, saturated real scenes) rather than random real photos.

- **JPEG re-encoding robustness**: Calibrate `color_gamut` and `screen_luminance` thresholds on re-encoded training copies.
- **Phone-native implementation**: Use accelerated frameworks (RenderScript/Vulkan on Android, Metal/Accelerate on iOS) to get <15ms latency.
- **Threshold tuning based on fraud cost**: Operate at higher recall if false negatives are 10–50× more costly than false positives.

Guidance on methodologies sourced from Sightengine image recapture detection docs. Assistance with implementation and debugging provided by Claude (Anthropic).